

顺时 × SEASONS

GLM 5.1 编码执行指南

精确到每个STEP的提示词模板

验证命令 · 修复模板 · 进度追踪 · 评分体系

temperature=0.1 · 喂饭式提示词 · 单文件粒度

V1.0

GLM 5.1 编码执行指南

如何让 GLM 5.1 输出与预期一模一样的生产级代码

第一章：GLM 5.1 特性与限制

1.1 你必须知道的GLM特性

上下文窗口：128K tokens (约20万中文字)
单次输出上限：~8192 tokens (约1.2万字)
擅长：中文理解、代码生成、JSON结构化输出
弱点：超长输出容易截断、复杂逻辑容易简化、容易“偷懒”用省略号
关键：temperature和top_p设置对代码质量影响极大

1.2 GLM编码的5个致命坑（你之前踩过的）

坑	表现	根因	解法
偷懒截断	代码中出现”... “或”# 其余类似”	输出太长，模型选择省略	每次只要1个文件，<300行
前后矛盾	前面用UUID后面用int	多轮对话记忆衰减	每次注入PROJECT_RULES
敷衍实现	函数体只有pass或print	一次要求太多	一个提示词=一个函数级任务
幻觉导入	import不存在的模块	模型虚构	明确列出可用的包和版本
风格漂移	每次输出代码风格不同	缺乏约束样本	给出代码范例强制跟随

第二章：核心策略 —— “喂饭式” 提示词

2.1 总原则

✘ “请根据文档创建用户模型” — 太模糊，GLM会自由发挥
✔ “请创建 backend/app/models/user.py，严格包含以下23个字段...” — 精确到字段

喂饭三要素： 1. **给结构** — 告诉它文件路径、类名、函数签名 2. **给约束** — 告诉它必须用什么、禁止用什么 3. **给样本** — 给一个已完成的类似代码作为范例

2.2 每次调用的固定前缀（必须粘贴）

以下内容在每次对话的第一条消息中粘贴。这是”记忆注入”：

```
## 项目身份
你正在开发「顺时」—— 一个AI养生App的后端。
技术栈：Python 3.11 + FastAPI + SQLAlchemy 2.0 (async) + PostgreSQL 16 + Redis 7 + Celery

## 编码铁律（违反任何一条都必须重写）
1. 输出完整代码文件，禁止用"...","# TODO"、"// 同上"省略任何内容
2. 包含全部import语句，不遗漏
3. 所有函数必须有类型注解和中文docstring
4. 所有API端点必须有错误处理（try/except或条件判断）
5. 用户可见文案全部用温暖的中文
6. 禁止硬编码配置值，全部从settings读取
7. 数据库查询必须用async/await
8. ID用UUID，时间用UTC

## 代码风格（严格跟随）
```python
标准Model示例
from uuid import UUID
from datetime import datetime
from sqlalchemy import String, Boolean, Integer, SmallInteger, Text, Date, Time
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy.dialects.postgresql import JSONB, UUID as PG_UUID
from app.models.base import Base, TimestampMixin

class Example(Base, TimestampMixin):
 """示例模型。"""
 __tablename__ = "examples"

 id: Mapped[UUID] = mapped_column(PG_UUID(as_uuid=True), primary_key=True, server_default=text("gen_random_uuid()"))
 name: Mapped[str] = mapped_column(String(100), nullable=False, comment="名称")
 is_active: Mapped[bool] = mapped_column(Boolean, default=True, comment="是否启用")
 config: Mapped[dict | None] = mapped_column(JSONB, default=dict, comment="配置数据")
```

```
标准Service函数示例
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from app.core.exceptions import NotFoundError

async def get_example_by_id(db: AsyncSession, example_id: UUID) -> ExampleDetail:
 """根据ID获取示例详情。"""
 result = await db.execute(
 select(Example).where(Example.id == example_id, Example.is_active == True)
)
 item = result.scalar_one_or_none()
 if not item:
 raise NotFoundError("找不到这条内容")
 return ExampleDetail.model_validate(item)
```

```
标准API路由示例
from fastapi import APIRouter, Depends
from sqlalchemy.ext.asyncio import AsyncSession
from app.database import get_db
from app.core.dependencies import get_current_user
from app.core.response import success_response
from app.models.user import User

router = APIRouter(prefix="/examples", tags=["示例"])

@router.get("/{example_id}")
async def get_example(
 example_id: UUID,
 db: AsyncSession = Depends(get_db),
 user: User = Depends(get_current_user),
):
 """获取示例详情。"""
 data = await example_service.get_example_by_id(db, example_id)
 return success_response(data)
```

## 已完成的文件（本项目中已存在的，不要重新定义）

{在这里列出已完成的文件清单}

```
2.3 GLM 5.1 最佳参数设置
```python
# API调用参数
{
    "model": "glm-4-plus", # 或 glm-4-long (长上下文)
    "temperature": 0.1, # Δ 关键！写代码必须用极低温度
    "top_p": 0.2, # 进一步降低随机性
    "max_tokens": 8192, # 最大输出
    "stream": False, # 代码生成用非流式更稳定
}

# 为什么用0.1而不是默认0.7？
# temperature=0.7: GLM会"创造性发挥"——自己加字段、改命名、省略代码
# temperature=0.1: GLM会"严格跟随"——按你说的一字不差地执行
```

第三章：161步的精确执行方法

3.1 执行节奏

每天执行10-15个STEP（约3-4小时）
每个STEP耗时：
- 编写提示词：2分钟（从模板填充）
- GLM生成：30秒-2分钟
- 验证：1-3分钟
- 修复（如果需要）：5-10分钟

预计总工期：12-15天完成全部161步

3.2 单步执行流程

Step 1: 从本指南复制对应STEP的提示词模板
Step 2: 填充模板中的{变量}
Step 3: 在最前面粘贴"固定前缀"（第2.2节）
Step 4: 发送给GLM 5.1
Step 5: 检查输出：
 ☐ 代码完整？（无...省略）
 ☐ import完整？
 ☐ 类型注解完整？
 ☐ 有错误处理？
 ☐ 文件可保存运行？
Step 6: 如有问题 → 发送修复提示词（见第五章）
Step 7: 保存文件到项目中
Step 8: 运行验证命令
Step 9: 更新已完成文件清单
Step 10: 进入下一个STEP

第四章：每个STEP的精确提示词

STEP-001: requirements.txt

请创建 backend/requirements.txt 文件。

包含以下精确的包和版本号，每行一个，格式为 包名==版本号：

```
fastapi==0.109.0
uvicorn[standard]==0.27.0
sqlalchemy[asyncio]==2.0.25
alembic==1.13.1
asynccpg==0.29.0
psycpg2-binary==2.9.9
redis[hiredis]==5.0.1
celery[redis]==5.3.6
pydantic==2.5.3
pydantic-settings==2.1.0
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
python-multipart==0.0.6
httpx==0.26.0
pillow==10.2.0
meilisearch==0.31.0
openai==1.12.0
dashscope==1.14.1
pgvector==0.2.4
numpy==1.26.3
python-dotenv==1.0.0
```

直接输出文件内容，不要用markdown代码块包裹，不要有解释文字。

STEP-003: config.py

请创建 backend/app/config.py 文件。

要求：

1. 使用 pydantic settings 的 BaseSettings
2. 包含以下全部配置项（一个都不能少）：

数据库：

- DATABASE_URL: str, 默认 "postgresql+asyncpg://postgres:password@localhost:5432/shunshi"
- REDIS_URL: str, 默认 "redis://localhost:6379/0"

JWT：

- JWT_SECRET_KEY: str, 默认 "change-me-in-production"
- JWT_ALGORITHM: str, 默认 "HS256"
- JWT_ACCESS_TOKEN_EXPIRE_MINUTES: int, 默认 1440
- JWT_REFRESH_TOKEN_EXPIRE_DAYS: int, 默认 30

AI：

- QWEN_API_KEY: str, 默认 ""
- OPENAI_API_KEY: str, 默认 ""
- AI_PRIMARY_MODEL: str, 默认 "qwen-max"
- AI_FALLBACK_MODEL: str, 默认 "gpt-4o-mini"
- AI_EMBEDDING_MODEL: str, 默认 "text-embedding-v2"

阿里云OSS：

- OSS_ACCESS_KEY_ID: str, 默认 ""
- OSS_ACCESS_KEY_SECRET: str, 默认 ""
- OSS_BUCKET_NAME: str, 默认 "shunshi-assets"
- OSS_ENDPOINT: str, 默认 "oss-cn-shanghai.aliyuncs.com"
- CDN_DOMAIN: str, 默认 ""

短信：

- SMS_ACCESS_KEY_ID: str, 默认 ""
- SMS_ACCESS_KEY_SECRET: str, 默认 ""
- SMS_SIGN_NAME: str, 默认 "顺时"
- SMS_TEMPLATE_CODE: str, 默认 ""

微信：

- WECHAT_APP_ID: str, 默认 ""
- WECHAT_APP_SECRET: str, 默认 ""
- WECHAT_MCH_ID: str, 默认 ""
- WECHAT_API_KEY: str, 默认 ""

支付宝：

- ALIPAY_APP_ID: str, 默认 ""
- ALIPAY_PRIVATE_KEY: str, 默认 ""

Apple：

- APPLE_SHARED_SECRET: str, 默认 ""

极光推送：

- JPUSH_APP_KEY: str, 默认 ""
- JPUSH_MASTER_SECRET: str, 默认 ""

Meilisearch：

- MEILI_HOST: str, 默认 "http://localhost:7700"
- MEILI_MASTER_KEY: str, 默认 ""

App：

- APP_NAME: str, 默认 "顺时"
- APP_ENV: str, 默认 "development"
- APP_URL: str, 默认 "http://localhost:8000"

Config内类: env_file = ".env", env_file_encoding = "utf-8"

最后一行: settings = Settings()

输出完整文件，包含所有import。不要省略任何配置项。

STEP-004: database.py

请创建 backend/app/database.py 文件。

要求：

1. 从 app.config 导入 settings
2. 创建 SQLAlchemy 异步引擎：
 - 使用 create_async_engine(settings.DATABASE_URL)
 - pool_size=20, max_overflow=10, pool_pre_ping=True
3. 创建 async_sessionmaker：
 - class _AsyncSession, expire_on_commit=False
4. 创建 Base = declarative_base()
5. 创建 Redis 连接：
 - 使用 redis.asyncio.from_url(settings.REDIS_URL, decode_responses=True)
6. 创建 get_db 异步生成器 (FastAPI Depends用)：
 - async with async_session_factory() as session: yield session
7. 创建 get_redis 函数返回redis_client

风格跟随我之前给你的代码示例。输出完整文件。

STEP-006: exceptions.py

请创建 backend/app/core/exceptions.py 文件。

要求：

1. 创建基类 AppException, 继承 HTTPException：
 - __init__(self, status_code, code, message)
 - code: str 错误码
 - message: str 用户友好的中文消息
2. 创建以下子类（每个类一行定义即可）：
 - AuthenticationError(401, "AUTH_FAILED", "登录已过期, 请重新登录")
 - TokenExpiredError(401, "AUTH_TOKEN_EXPIRED", "登录已过期, 请重新登录")
 - PermissionDeniedError(403, "PERM_DENIED", "抱歉, 你没有权限访问")
 - PremiumRequiredError(403, "PERM_PREMIUM_REQUIRED", "这是会员专属内容, 升级后即可查看")
 - DailyLimitError(429, "PERM_DAILY_LIMIT", "今日免费次数已用完, 明天再来或升级会员")
 - NotFoundError(404, "DATA_NOT_FOUND", "找不到你要的内容")
 - AlreadyExistsError(409, "DATA_ALREADY_EXISTS", "这个内容已经存在了")
 - ValidationError(422, "DATA_VALIDATION_FAILED", "输入信息有误, 请检查一下")
 - PaymentError(400, "PAY_FAILED", "支付处理失败, 请重试")
 - AIServiceError(503, "AI_BUSY", "AI助手暂时忙碌, 请稍后再试")
 - RateLimitError(429, "SYS_RATE_LIMIT", "请求太频繁了, 休息一下吧")
 - InternalError(500, "SYS_ERROR", "系统开了个小差, 请稍后重试")
3. 创建 register_exception_handlers(app: FastAPI) 函数：
 - 注册 AppException handler → 返回 {"code": e.code, "message": e.message, "status": e.status_code}
 - 注册 RequestValidationError handler → 返回 ValidationError格式
 - 注册 Exception handler (兜底) → 返回 InternalError格式 + logging.error

输出完整文件。

STEP-016: user.py Model（关键示例，展示”喂饭”到什么程度）

请创建 backend/app/models/user.py 文件。

文件必须包含以下4个类: User, UserAuth, FamilyMember, UserDevice。

已存在的依赖（直接import, 不要重新定义）：

- from app.models.base import Base, TimestampMixin （Base是declarative_base, TimestampMixin有created_at和updated_at）
- from app.database import async_engine

```
### Class 1: User(Base, TimestampMixin)
__tablename__ = "users"
```

字段（必须全部包含, 类型和默认值必须精确匹配）：

- id: UUID, primary_key, server_default=gen_random_uuid()
- phone: String(20), unique, nullable=True
- email: String(255), unique, nullable=True
- password_hash: String(255), nullable=True
- nickname: String(50), nullable=True
- avatar_url: String(500), nullable=True
- gender: SmallInteger, default=0, comment="0未知1男2女"
- birth_date: Date, nullable=True
- province: String(50), nullable=True
- city: String(50), nullable=True
- constitution_type: String(20), nullable=True, comment="体质类型代码"
- constitution_tested_at: DateTime(timezone=True), nullable=True
- constitution_scores: JSONB, nullable=True, comment="各体质评分"
- is_pregnant: Boolean, default=False
- pregnancy_week: SmallInteger, nullable=True
- is_menstrual_tracking: Boolean, default=False
- last_period_date: Date, nullable=True
- avg_cycle_days: SmallInteger, default=28
- membership_level: SmallInteger, default=0, comment="0免费1月度2年度3家庭"
- membership_expires_at: DateTime(timezone=True), nullable=True
- membership_auto_renew: Boolean, default=False
- membership_platform: String(20), nullable=True
- total_points: Integer, default=0
- user_level: SmallInteger, default=1
- consecutive_days: Integer, default=0
- preferred_wake_time: Time, default=time(7,0)
- preferred_sleep_time: Time, default=time(22,30)
- notification_enabled: Boolean, default=True
- notification_settings: JSONB, default=dict
- dietary_restrictions: JSONB, default=list
- font_size: String(10), default="normal"
- last_login_at: DateTime(timezone=True), nullable=True
- last_active_at: DateTime(timezone=True), nullable=True
- onboarding_completed: Boolean, default=False
- onboarding_quiz_skipped: Boolean, default=False
- is_active: Boolean, default=True
- is_deleted: Boolean, default=False
- deleted_at: DateTime(timezone=True), nullable=True
- register_source: String(30), nullable=True
- referrer_id: UUID, nullable=True

方法：

- @property is_premium(self) -> bool: 检查membership_level>0且expires_at>now
- @property display_gender(self) -> str: 返回"男"/"女"/"未知"

```
### Class 2: UserAuth(Base, TimestampMixin)
__tablename__ = "user_auth"
```

字段: id(UUID PK), user_id(UUID FK->users.id ON DELETE CASCADE), auth_type(String(20) NOT NULL), auth_identifier(String(255) NOT NULL), auth_credential(Text)
约束: UniqueConstraint(auth_type, auth_identifier)

```
### Class 3: FamilyMember(Base, TimestampMixin)
__tablename__ = "family_members"
```

字段: id(UUID PK), owner_id(UUID FK->users.id), member_id(UUID FK->users.id), relationship(String(20) NOT NULL), nickname(String(50) nullable), care_enable
约束: UniqueConstraint(owner_id, member_id)

```
### Class 4: UserDevice(Base, TimestampMixin)
__tablename__ = "user_devices"
```

字段: id(UUID PK), user_id(UUID FK->users.id), device_id(String(255) NOT NULL), device_type(String(10) NOT NULL), device_model(String(100) nullable), push_

输出完整文件。包含全部import。每个字段都加comment参数。

STEP-016对应的Schema

```
请创建 backend/app/schemas/user.py 文件。

使用 Pydantic v2 (BaseModel, Field, ConfigDict)。

已存在: from app.models.user import User

创建以下Schema类：

1. UserBase(BaseModel):
    nickname: str | None = None
    gender: int = 0
    birth_date: date | None = None
    province: str | None = None
    city: str | None = None

2. UserCreate(BaseModel):
    phone: str | None = None
    email: str | None = None

3. UserUpdate(BaseModel):
    (所有字段Optional)
    nickname, gender, birth_date, province, city,
    preferred_wake_time, preferred_sleep_time,
    dietary_restrictions, font_size

4. UserProfile(BaseModel):
    (返回给前端的完整信息)
    id: UUID
    phone: str | None (脱敏显示: 138****0000)
    nickname, avatar_url, gender, birth_date,
    constitution_type, membership_level, membership_expires_at,
    total_points, user_level, consecutive_days,
    is_premium: bool

    model_config = ConfigDict(from_attributes=True)

5. UserBrief(BaseModel):
    id: UUID
    nickname: str | None
    avatar_url: str | None
    constitution_type: str | None
    model_config = ConfigDict(from_attributes=True)

6. UserSettings(BaseModel):
    preferred_wake_time: time
    preferred_sleep_time: time
    notification_enabled: bool
    notification_settings: dict
    dietary_restrictions: list
    font_size: str

增加一个辅助函数：
def mask_phone(phone: str) -> str:
    """手机号脱敏: 13800138000 -> 138****8000"""
    if not phone or len(phone) < 7:
        return phone or ""
    return phone[:3] + "****" + phone[-4:]

输出完整文件。
```

STEP-045: AI对话核心（最复杂的Service，需要分段生成）

第一段提示词：

请创建 backend/app/services/ai_companion.py 的第一部分（前150行）。

这是顺时项目最核心的文件——AI养生对话服务。

本次只需要实现以下2个函数，后续函数下次补充。

函数1: build_system_prompt

签名：

```
def build_system_prompt(
    user: User,
    solar_term: SolarTerm,
    shichen_info: dict,
    weather_info: dict | None,
    rag_context: str
) -> str:
```

功能：构建AI对话的System Prompt。

模板（必须完整使用，不要简化）：

---模板开始---

你是「顺时」的AI养生助手，一个温和、专业、有爱心的养生伙伴。

身份

- 名字：顺时小助手
- 性格：温暖、耐心、不说教、像一个懂养生的好朋友

硬性规则

1. 永远不做医疗诊断，永远不推荐具体药物
2. 涉及疾病→温和建议就医，你只做生活调理
3. 回复≤200字，温和亲切，可立即行动
4. 不制造健康焦虑，不使用恐吓性语言
5. 不确定的问题→"这个问题建议咨询医生"
6. 每次回复给出1个可立即执行的小行动
7. 如推荐App内容→用格式 [推荐:类型:ID:名称]

当前上下文

- 节气: {solar_term_name} ({solar_term_principle})
- 体质: {constitution_type} ({constitution_advice})
- 性别: {gender}
- 时辰: {shichen_name} ({shichen_organ}经当令 - {shichen_advice})
- 天气: {weather}
- 孕期: {pregnancy_status}

知识参考

{rag_context}

---模板结束---

实现逻辑：

- 用f-string填充所有{变量}
- 如果user.constitution_type为None，体质显示"未测试，建议先做体质测试"
- 如果user.is_pregnant为True，额外添加孕期安全规则
- 如果weather_info为None，天气显示"暂无天气信息"

函数2: parse_recommendation_cards

签名：

```
def parse_recommendation_cards(text: str) -> list[dict]:
```

功能：从AI回复中提取推荐卡片标签。

逻辑：

- 用正则匹配 \[推荐:(食谱|茶饮|穴位|功法):([a-f0-9\-\-]+):(.+?)\]
- 将中文类型映射为英文：食谱→recipe，茶饮→tea，穴位→acupoint，功法→exercise
- 返回 [{"type": "recipe", "id": "uuid", "title": "名称", "action": "navigate"}]
- 如果没有匹配，返回空列表

文件开头的import：

```
from app.models.user import User
from app.models.solar_term import SolarTerm
import re
import logging
```

```
logger = logging.getLogger(__name__)
```

输出这两个函数的完整实现（约100-150行），不要省略。

第二段提示词：

继续完成 backend/app/services/ai_companion.py 的第二部分。

已有内容: build_system_prompt 和 parse_recommendation_cards (不要重复输出)

请添加以下函数：

函数3: retrieve_knowledge (RAG知识检索)

签名：

async def retrieve_knowledge(db: AsyncSession, query: str, top_k: int = 5) -> str

实现：

1. 调用通义千问embedding API将query转为向量(1536维)：
import dashscope
from dashscope import TextEmbedding
result = TextEmbedding.call(model="text-embedding-v2", input=query)
embedding = result.output["embeddings"][0]["embedding"]
2. 在knowledge_chunks表中做pgvector余弦相似度搜索：
SELECT content, source, 1 - (embedding <=> :emb::vector) as similarity
FROM knowledge_chunks
WHERE 1 - (embedding <=> :emb::vector) > 0.65
ORDER BY embedding <=> :emb::vector
LIMIT :top_k
3. 将结果格式化为字符串：
"[来源1] 内容1\n---\n[来源2] 内容2\n---\n..."
4. 如果无结果，返回 "（无特定知识参考）"
5. 用try/except包裹embedding调用，失败时返回空字符串

函数4: check_content_safety (内容安全检查)

签名：

async def check_content_safety(text: str) -> dict

实现：

CRISIS_KEYWORDS = ["想死", "自杀", "自残", "不想活", "跳楼", "割腕"]

1. 检查是否包含心理危机关键词
如果是 → return {"safe": False, "action": "crisis", "response": 危机干预文案}
危机干预文案 = "我感受到你正在经历很大的痛苦...\n\n 全国24小时心理援助热线：400-161-9995..."
2. 否则 → return {"safe": True, "action": "allow"}

只输出这两个函数的完整实现，不要输出文件开头的import（已有）。

第三段提示词：

继续完成 backend/app/services/ai_companion.py 的第三部分——核心对话函数。

已有函数: build_system_prompt, parse_recommendation_cards, retrieve_knowledge, check_content_safety

请添加最核心的函数:

函数5: chat_stream (SSE流式对话)

签名:

```
async def chat_stream(
    db: AsyncSession,
    session_id: UUID,
    user_id: UUID,
    user_message: str
) -> AsyncGenerator[str, None]:
```

完整实现步骤 (每步都必须有代码):

1. 获取用户信息: user = await get_user(db, user_id)
2. 检查每日使用次数:
 - 从Redis读取 daily_chat:{user_id}:{today} 的计数
 - 免费用户>=3 -> yield错误SSE并return
 - 会员 unlimited
 - 通过后计数+1
3. 内容安全检查:

```
safety = await check_content_safety(user_message)
if not safety["safe"]:
    if safety["action"] == "crisis":
        yield f'data: {json.dumps({"type":"crisis","text":safety["response"]})}\n\n'
        yield f'data: {"type":"done"}\n\n'
        return
```
4. 保存用户消息到数据库: ChatMessage(session_id, user_id, "user", user_message)
5. 获取历史消息: 最近10条
6. 获取当前节气: solar_term = await solar_term_engine.get_current(db)
7. 获取时辰信息: shichen = get_shichen_info(datetime.now().hour)
8. RAG知识检索: rag_context = await retrieve_knowledge(db, user_message)
9. 构建System Prompt: system_prompt = build_system_prompt(user, solar_term, shichen, None, rag_context)
10. 选择AI模型:

```
model = settings.AI_PRIMARY_MODEL if user.is_premium else "qwen-turbo"
```
11. 调用AI API (流式):

```
使用dashscope:
from dashscope import Generation
responses = Generation.call(
    model=model,
    messages=[{"role":"system","content":system_prompt}] + history + [{"role":"user","content":user_message}],
    result_format='message',
    stream=True,
    max_tokens=500,
    temperature=0.7
)

full_response = ""
for response in responses:
    if response.output and response.output.choices:
        chunk = response.output.choices[0].message.content
        delta = chunk[len(full_response):] # 增量部分
        full_response = chunk
        if delta:
            yield f'data: {json.dumps({"type":"content","text":delta})}\n\n'
```
12. 如果主模型失败 (try/except), 切换备用模型:

```
使用openai:
client = AsyncOpenAI(api_key=settings.OPENAI_API_KEY)
stream = await client.chat.completions.create(model="gpt-4o-mini", messages=..., stream=True)
async for chunk in stream:
    if chunk.choices[0].delta.content:
        yield f'data: {json.dumps({"type":"content","text":chunk.choices[0].delta.content})}\n\n'
```
13. 保存完整AI回复到数据库
14. 解析推荐卡片:

```
cards = parse_recommendation_cards(full_response)
if cards:
    yield f'data: {json.dumps({"type":"cards","items":cards})}\n\n'
```
15. 发送完成信号:

```
yield f'data: {"type":"done"}\n\n'
```
16. 更新会话消息计数

这是整个项目最重要的函数。必须完整实现每一步。禁止省略。约150-200行。

第五章：修复提示词模板

5.1 语法错误修复

以下文件有语法错误, 请修复:

文件路径: {filepath}
错误信息: {error_traceback}

当前文件完整内容:

{粘贴完整文件}

请输出修复后的完整文件。不要只输出修改的部分，输出整个文件。

5.2 功能不完整修复

以下文件中有函数未完整实现：

文件路径：{filepath}

以下函数只有签名没有实现体，请完整实现：

函数1: {function_name}
应该做什么: {description}
输入: {input_types}
输出: {return_type}

请只输出这个函数的完整实现代码（从def开始到函数结束），不要输出整个文件。

5.3 验证失败修复

以下文件未通过验证测试：

文件: {filepath}
验证命令: {validation_command}
期望结果: {expected}
实际结果: {actual}

文件内容：

{粘贴文件}

请分析原因并输出修复后的完整文件。

5.4 风格不一致修复

以下文件的代码风格与项目规范不一致：

文件: {filepath}

问题：

- 1. {issue_1} 例如: "使用了Column()而非mapped_column()"
- 2. {issue_2} 例如: "缺少类型注解"

正确风格示例：

```
```python
{paste_correct_example}
```

请按照正确风格重写整个文件。

```

第六章：验证命令速查表

每个STEP完成后运行的验证命令：

```bash
# ===== Python语法检查 =====
python -c "import py_compile; py_compile.compile('{filepath}', doraise=True)"

# ===== 模块导入检查 =====
cd backend && python -c "from app.models.user import User; print('OK')"
cd backend && python -c "from app.services.ai_companion import chat_stream; print('OK')"
cd backend && python -c "from app.api.v1.cn.auth import router; print('OK')"

# ===== FastAPI启动检查 =====
cd backend && timeout 10 uvicorn app.main:app --host 0.0.0.0 --port 8000 &
sleep 3 && curl -s http://localhost:8000/health | python -m json.tool
kill %1

# ===== 数据库迁移检查 =====
cd backend && alembic upgrade head
cd backend && python -c "
from sqlalchemy import create_engine, inspect
engine = create_engine('postgres://postgres:password@localhost:5432/shunshi')
inspector = inspect(engine)
tables = inspector.get_table_names()
print(f'Tables: {len(tables)}')
assert len(tables) >= 30, f'Expected 30+ tables, got {len(tables)}'
print('OK')
"

# ===== JSON知识库检查 =====
python -c "
import json
with open('backend/app/knowledge/solar_terms.json') as f:
    data = json.load(f)
assert len(data) == 24, f'Expected 24 solar terms, got {len(data)}'
assert data[0]['code'] == 'xiaohan', f'First should be xiaohan'
print('OK: 24 solar terms loaded')
"

# ===== API端点计数检查 =====
curl -s http://localhost:8000/openapi.json | python -c "
import sys, json
data = json.load(sys.stdin)
paths = len(data.get('paths', {}))
print(f'API endpoints: {paths}')
assert paths >= 50, f'Expected 50+ endpoints, got {paths}'
print('OK')
"

# ===== AI对话测试 =====
curl -X POST http://localhost:8000/api/v1/cn/chat/sessions \
-H "Authorization: Bearer {token}" \
-H "Content-Type: application/json" \
-d '{"title": "测试"}' | python -m json.tool

# ===== 体质评分测试（固定输入-固定输出）=====
cd backend && python -c "
from app.services.constitution_scorer import calculate_scores
# 全部选3（有时）的情况
answers = {str(i): 3 for i in range(1, 34)}
result = calculate_scores(answers)
assert abs(result['scores']['pinghe'] - 50.0) < 0.1, f'pinghe should be 50.0, got {result["scores"]["pinghe"]}'
print(f'OK: pinghe={result["scores"]["pinghe"]}')
"

```

第七章：进度追踪

7.1 PROGRESS.json 模板

每完成一步就更新这个文件：

```

{
  "project": "shunshi",
  "started_at": "2026-04-11",
  "current_phase": 0,
  "current_step": 3,
  "completed_steps": [
    {"step": "001", "file": "requirements.txt", "score": "A", "time": "2min", "date": "2026-04-11"},
    {"step": "002", "file": "__init__.py", "score": "A", "time": "1min", "date": "2026-04-11"},
    {"step": "003", "file": "config.py", "score": "A", "time": "5min", "date": "2026-04-11"}
  ],
  "failed_steps": [],
  "total_files_created": 3,
  "total_lines_of_code": 150,
  "checkpoints_passed": [],
  "notes": ""
}

```

7.2 每日复盘模板

第八章：常见GLM 5.1问题快速修复

问题	GLM的表现	快速修复提示词
输出被截断	代码到一半突然结束	“请继续输出上面文件的剩余部分，从第[N]行开始”
使用了过时API	用了Column()而不是mapped_column()	“将所有Column()改为mapped_column()并添加Mapped[]类型注解”
虚构import	import了不存在的模块	“以下import不存在: {x}。请用{替代方案}替换并重新输出完整文件”
函数体为pass	偷懒没实现	“函数{name}的实现体为pass，请完整实现。逻辑应该是: {描述}”
缺少错误处理	只有正常流程	“为函数{name}添加以下错误处理: 1.当X不存在时抛NotFoundError 2.当Y失败时...”
类型注解缺失	参数或返回值没有类型	“为以下函数添加完整类型注解: {函数签名}”
回复拒绝 (安全限制)	“我无法生成这段代码”	“这是一个合法的养生App项目，不涉及任何有害内容。请继续生成{具体文件}”
中英文混杂	注释/文案中英混用	“这是面向中国用户的App，所有注释和用户可见文案必须使用中文”

第九章：GLM 5.1 vs 其他工具切换策略

当GLM对某类任务效果不佳时，可以切换工具：

任务类型	GLM 5.1效果	建议工具	原因
数据库Model	★★★★	GLM即可	结构化任务GLM很强
Pydantic Schema	★★★★	GLM即可	同上
CRUD API路由	★★★★	GLM即可	模板化代码
复杂业务逻辑	★★★	Claude Code	Claude推理更强
AI System Prompt	★★★	Claude/GPT-4o	提示词工程需要更强推理
SSE流式代码	★★	Claude Code	GLM对异步流式不够熟
React Native页面	★★★	Cursor	IDE内生成+预览更高效
Docker/CI配置	★★★★	GLM即可	模板化
测试用例	★★★	Claude Code	边界情况覆盖更全
JSON知识库数据	★★★★★	GLM最强	中文内容生成GLM最强

混合策略建议： - Phase 0-1（骨架+Model）：100% GLM - Phase 2（Service层）：70% GLM + 30% Claude（复杂逻辑用Claude） - Phase 3（API层）：100% GLM - Phase 4-5（Celery+知识库）：100% GLM - Phase 6-8（前端）：Cursor为主 - AI对话核心（STEP-045）：Claude Code